

주짓수 관원 여러 명의 정보를 메모리에 상시 유지하고, 2차 과제와 성별/체급별 복합 조건문과 3차 과제의 메뉴 루프를 배열 구조에 맞춰 논리적 충돌 없이 '배열 전달 함수' 형태로 모듈화하는 아키텍처 설계 요구

4차 과제(V4.0)의 최종 목표인 **다중 데이터 배열 구조와 함수 모듈화 아키텍처**에 대한 설계 요약입니다.

단일 관원만 처리하던 이전 버전과 달리, 4차 과제에서는 데이터가 **배열**로 확장되면서 변수의 메모리 참조 방식과 함수의 데이터 전달 메커니즘이 완전히 새롭게 재구성됩니다. 이를 논리적 충돌 없이 설계하기 위한 핵심 아키텍처를 가이드해 드립니다.

주짓수 시스템 V4.0 데이터 흐름 및 모듈러 아키텍처

V4.0 시스템은 주소 참조(Call by Reference) 효과를 내는 **배열 매개변수 전달** 방식을 기반으로 제어부와 연산부, 출력부를 완벽하게 격리합니다.

[main 함수 (제어부)]

- └─ 변수 저장소: 각 데이터별 1차원/2차원 배열 공간 확보 (MAX_MEMBERS)
- └─ 메뉴 루프: while(1) 무한 루프 제어
 - └─ [메뉴 1 선택] → input_members(배열 주소들) 호출
 - └─ 사용자가 입력한 데이터가 메인 배열에 직접 기록 (포인터 효과)
 - └─ [메뉴 2 선택] → calculate_all(배열 주소들) 호출
 - └─ for 루프 순회하며 수식 연산 후 결과 배열 업데이트
 - └─ [메뉴 3 선택] → print_all_reports(배열 주소들) 호출
 - └─ 2차 중첩 IF문(체급) + 3차 논리연산자(승급) 결합 후 표(Table) 출력

1. 매개변수를 통한 배열 주소 전달 (Call by Address)

- 설계 논리: C언어에서 함수에 일반 변수를 넘기면 값이 복사되지만, 배열의 이름을 넘기면 배열의 첫 번째 요소의 주소가 전달됩니다.
- 공학적 이점: `input_members` 나 `calculate_all` 함수 내에서 배열의 값을 변경하면, `main` 함수에 있는 원본 배열 데이터가 실시간으로 직접 수정됩니다. 이로 인해 불필요한 메모리 복사 연산을 줄이고 다중 데이터를 안전하게 유지관리(상태 동기화)할 수 있습니다.

2. 기호 상수(`#define`) 기반의 가변형 루프 설계

- 코드 내부에서 인덱스 한계값으로 숫자 `3` 을 하드코딩하지 않고, `#define MAX_MEMBERS 3` 상수를 활용하여 모든 함수의 `for` 반복문 제어 조건(`i < MAX_MEMBERS`)을 통일했습니다.
- 이는 프로그램의 결합도(Coupling)를 낮추고 응집도(Cohesion)를 높이는 전형적인 모듈화 기법입니다.

3. 2차원 문자 배열을 통한 문자열(String) 격리

- 관원의 이름을 받기 위해 `char member_names[MAX_MEMBERS][20]` 구조를 도입했습니다.
- 행(Row) 인덱스는 관원의 고유 번호(ID)가 되고, 열(Column) 공간은 20바이트 크기의 영문 문자열 공간이 되어, `scanf("%s", member_names[i])` 호출 시 각 관원의 이름 데이터가 메모리 공간을 침범하지 않고 안전하게 격리 저장됩니다.

🎯 기능적 분리(Separation of Concerns) 명세

모듈(함수)명	입력 매개변수 (주소 참조)	내부 제어 및 결합 로직	아키텍처적 책무
<code>main</code>	시스템 내부 변수	<code>while(1)</code> 메뉴 인터페이스 및 배열 메모리 할당	프로세스 수명 관리 및 중앙 컨트롤러
<code>input_members</code>	이름, 성별, 체중, 기록 배열 전체	<code>for</code> 루프를 이용한 다중 스캔 및 입력 버퍼 관리	외부 환경으로부터 수집 및 배열 무